

第2章 数据的常量和变量表示

2.2 常量表达

2.3 变量表达

教科书第2章2.2与2.3



2.2

汇编语言的常量表达

常量 (Constant)

- 常量是程序中使用的一个确定数值
 - ▶ 在汇编阶段就可以确定，直接编码于指令代码中
 - ▶ 不是保存在存储器中可变的变量
- 汇编语言支持多种常量的表达形式
 - ▶ 常数：直接给出数值本身
 - ▶ 字符和字符串：使用ASCII码值
 - ▶ 符号常量：用标识符替代数值
 - ▶ 数值表达式：经过运算得到数值

MASM 常量

- 十、十六和二进制形式表达的数值
 - ▶ 以后缀字母（**D**、**H**和**B**）区分，十进制数可以不加
- 以字母A~F开头的十六进制常数，要加前导0
 - ▶ 以便区别于以字母开头的标识符

“AH”是指寄存器 “0AH”表示十六进制常数（即十进制常数10）

十进制数	100, 255D
十六进制数	64H, 0FFH, 0B800H
二进制数	01101100B

MASM 常量表达：字符和字符串

- 英文缩略号（单引号）或双引号括起来的单个或多个字符
- 每个字符的数值是对应的ASCII码值

字符	'd' (=64H)
字符串	'Hello, Assembly !' "Let's have a try." 'Say "Hello", mybaby.'

单引号、双引号没有区别

字符和字符串也没有本质上的区别

MASM 常量表达：符号常量

- 使用标识符表达一个数值 符号名 = 数值表达式； 符号名
`equ` 数值表达式； 符号名 `equ` <字符串>

等号=	STD_INPUT_HANDLE = -10 STD_OUTPUT_HANDLE = -11
等价EQU	NULL <code>equ</code> 0 WriteConsole <code>equ</code> < WriteConsoleA>

MASM 常量表达：数值表达式

➤ 用运算符连接各种常量构成的算式

▶ 常用的算术运算符：

$+$ （加） $-$ （减） $*$ （乘） $/$ （除）

➤ 数值用作地址，数值表达式成为地址表达式

▶ 地址表达式只能使用加减

▶ 指示地址移动常量表示的若干个字节存储单元

地址 $+$ 常量

地址 $-$ 常量

算术表达式	67h-52h
地址表达式	var+4

常量表达

1. 常数

- ▶ 十、十六和二进制形式表达的数值

2. 字符和字符串

- ▶ 英文单引号或双引号括起的单个或多个字符

3. 符号常量

- ▶ 使用标识符表达一个数值

4. 数值表达式

- ▶ 用运算符连接各种常量构成的算式

值

常量具有确定数

常量的数据表达程序

常量表达

1. 常数

- ▶ 十、十六和二进制形式表达的数值

2. 字符和字符串

- ▶ 英文单引号或双引号括起的单个或多个字符

3. 符号常量

- ▶ 使用标识符表达一个数值

4. 数值表达式

- ▶ 用运算符连接各种常量构成的算式

常量具有确定数值

数据表达程序—1

00000000

64 64 64 64 64

机器指令

const1 byte 100,100d,01100100b,64h,'d'

相对地址

汇编语句

不同进制和形式表达了同一个数值：100 (=64H)
无论源程序如何表达，计算机内部都是二进制编码

(8位) 字节变量定义伪指令：byte

数据表达程序—2

00000005 01 7F 80 80 FF FF

const2 byte 1,+127,128,-128,255,-1

0000000B 69 97 20 E0 32 CE

const3 byte 105,-105,32,-32,32h,-32h

无符号数255和有符号数-1: **FFH**

无符号数128和有符号数-128: **80H**

105的补码: **69H**, -105的补码: **97H**

数据表达程序—3

00000011 30 31 32 33 34 35 36 37 38 39
6162 63 78 79 7A
41 42 43 58 59 5A

const4 byte '0123456789','abcdefghijklmnopqrstuvwxyz','ABCXYZ'

定义字符串：是每个字符的ASCII码
(例如0是30，Z是5A)

数据表达程序—4

00000027 0D 0A 00

crlf byte 0dh,0ah,0

数值0DH（回车符）和0AH（换行符）

注意前导零不能省略（否则成为DH和AH）

数字“0”用于字符串结尾

数据表达程序—5

= 0000000A minint = 10

= 000000FF maxint equ 0ffh

0000002A 0A 0F FA F5

const5 byte minint,minint+5

byte maxint-5,maxint-minint

符号MININT为10，MAXINT为255，不占主存空间
表达式定义CONST5的每个参数，实质还是常量

数据表达程序—6

0000002E 10 56 15 EB

const6 byte 4*4,34h+34,67h-52h,52h-67h

表达式定义CONST6的每个参数，但实质还是常量

mov eax,offset const4 ;从CONST4开始显示
call dispmsg

```
D:\MASM>make32 eg0201
```

```
D:\MASM>eg0201.exe
```

操作过程

运行结果

0123456789abcdefghijklmnopqrstuvwxyzABCXYZ

数据表达程序

;数据段

const1 **byte 100,100d,01100100b,64h,'d'**

const4 **byte '0123456789','abcxyz','ABCXYZ'**

crlf **byte 0dh,0ah,0**

运行结果

0123456789abcxyzABCXYZ

;代码段

mov eax,offset const4

call dispmsg

2.3

汇编语言的变量定义

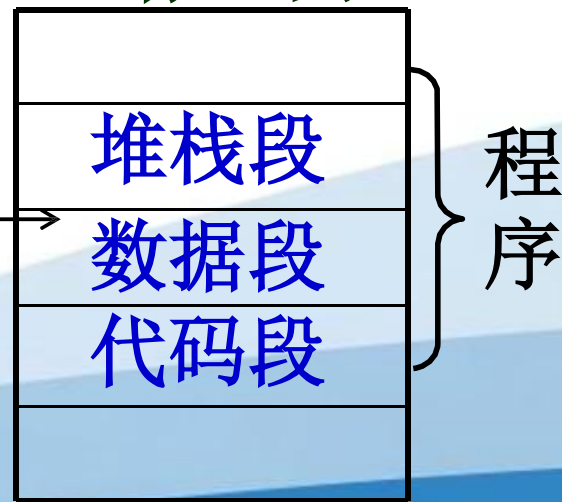
什么是变量

➤ 变量 (Variable)

- ▶ 随程序运行会发生变化的数据
 - ▶ 保存在可读可写的主存空间
- 变量的实质是主存单元的数据，因而可以改变
- ▶ 变量需要事先定义才能使用
 - ▶ 变量具有属性方便应用

变量表达主存数据，即存储器操作数

主存空间





变量的定义

变量名

变量定义伪指令

初值表

变量名是用户标识符，表示首元素逻辑地址

伪指令助记符：`byte`、`word`、`dword`，表示变量类型

初值表是用逗号分隔的一个或多个参数，表示变量初值

主要的变量定义伪指令

助记符: BYTE 变量类型: 字节	分配一个或多个字节单元 每个数据是8位、字节量
助记符: WORD 变量类型: 字	分配一个或多个字单元 每个数据是16位、字量
助记符: DWORD 变量类型: 双字	分配一个或多个双字单元 每个数据是32位、双字量

对应C语言变量类型: **char** **short** **long**

变量定义的初值表

- 变量定义是申请存储空间
 - ▶ 同时还可以进行存储单元初始化
 - ▶ 即用初值表赋予变量初值
- 初值表可以有一个或多个参数，多个参数用逗号分隔
 - ▶ 各种形式的常量
 - ▶ 使用“？”表示初值不确定，即未赋初值
 - ▶ 使用复制操作符**DUP**表示多个同样数值

复制操作符格式： 重复次数 **DUP**(重复参数)

本讲总结

➤ 变量需要先定义才能使用

➤ 变量定义的格式

变量名 变量定义伪指令 参数, 参数

➤ 变量定义之后

- ▶ 分配了存储空间
- ▶ 赋予了初值（也可以没有初值）
- ▶ 创建了变量名

（可以获知变量的地址和类型）



8位变量定义

汇编语言的变量定义

➤ 变量需要先定义才能使用

➤ 变量定义的格式

变量名 变量定义伪指令 参数, 参数

➤ 变量定义之后

- 分配了存储空间
 - 赋予了初值（也可以没有初值）
 - 创建了变量名
- （可以获知变量的地址和类型）



主要的变量定义伪指令

助记符: BYTE 变量类型: 字节	分配一个或多个字节单元 每个数据是8位、字节量
助记符: WORD 变量类型: 字	分配一个或多个字单元 每个数据是16位、字量
助记符: DWORD 变量类型: 双字	分配一个或多个双字单元 每个数据是32位、双字量

对应C语言变量类型: **char** **short** **long**

BYTE: 定义字节量(8位)数据的变量

- 8位无符号整数: $0 \sim 255$
- 8位补码表示的有符号整数: $-128 \sim +127$
- 一个字符 (ASCII码值)
- 压缩BCD码: $0 \sim 99$
- 非压缩BCD码: $0 \sim 9$
-

8位 (字节量、Byte-sized)

msg	byte	'Hello',13,10,0
const1	byte	100,64h,'d'
const6	byte	4*4,34h+34

定义字符串要使用字节变量定义BYTE

字节变量程序—1

00000000

00 80 FF 80 00 7F

bvar1 byte 0,128,255,-128,0,+127

相对地址

汇编语句

机器指令

数据段

7FH
00H
80H
FFH
80H
00H

BYTE伪指令定义的每个数据都是8位、1个字节

+1
bvar1

字节变量程序一2

00000006

01 FF 26 DA 38 C8

bvar2 byte 1,-1,38,-38,38h,-38h

38和-38，十进制数
38H和-38H，十六进制数

数据段

C8H
38H
DAH
26H
FFH
01H



字节变量程序—3

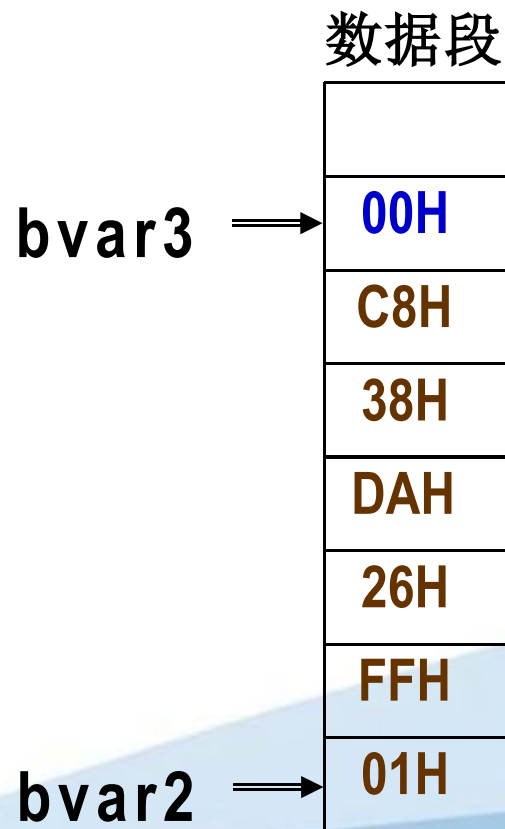
00000006 01 FF 26 DA 38 C8

bvar2 byte 1,-1,38,-38,38h,-38h

0000000C 00

bvar3 byte ?

变量BVAR3无初值 只在
主存保留存储空间
实际上用0填充



字节变量程序一

0000000D 00000005 [24]

bvar4 byte 5 dup ('\$')

变量BVAR4定义了5个相同的数据

复制操作符格式： 重复次数 DUP(重复参数)

数据段

bvar4



bvar3



24H
24H
24H
24H
24H
00H

字节变量程序—5

=0000000A

minint = 10

00000012

0000000A [00] 0000000A [0A 00]

bvar5 byte minint dup(0),minint dup(minint,?)

实际上就是

bvar5 byte 10 dup(0),10 dup(10,?)

符号常量使用等价的数值替代
变量BVAR5定义了10个0，再10个10和0

数据段

00H

0AH

00H

0AH

00H

00H

10个

10个

bvar5

字节变量程序—6

00000030

00000002 [02 03 00000002 [04]]

byte 2 dup(2,3,2 dup(4))

复制操作符DUP可以嵌套 主存中分配了
变量初值，但没有变量名

复制操作符格式： 重复次数 DUP(重复参数)

数据段

04H
04H
03H
02H
04H
04H
03H
02H

本讲总结

- 8位变量定义，使用**byte**伪指令
 - 每个数据是一个字节量，占用一个存储单元
- 字符（串）变量定义也使用**byte**伪指令
 - 每个字符是一个8位**ASCII**码
- 变量定义的参数不区别有无符号
 - 可以是无符号数，也可以是有符号数

对应C语言变量类型：**char**

16位变量定义

汇编语言的变量定义

➤ 变量需要先定义才能使用

➤ 变量定义的格式

变量名 变量定义伪指令 参数, 参数

➤ 变量定义之后

- 分配了存储空间
- 赋予了初值（也可以没有初值）
- 创建了变量名

（可以获知变量的地址和类型）



主要的变量定义伪指令

助记符: BYTE 变量类型: 字节	分配一个或多个字节单元 每个数据是8位、字节量
助记符: WORD 变量类型: 字	分配一个或多个字单元 每个数据是16位、字量
助记符: DWORD 变量类型: 双字	分配一个或多个双字单元 每个数据是32位、双字量

对应C语言变量类型: **char** **short** **long**

WORD: 定义字量(16位)数据的变量

- 16位无符号整数: $0 \sim 65535$
- 16位补码表示的有符号整数: $-32768 \sim +32767$
- 16位段地址
- 16位偏移地址
-

16位 (字量、Word-sized)

```
wvar1  word 0,-32768,65535  
wvar2  word ?  
minint = 10  
wvar3  word 5 dup(minint)
```

字变量程序—1

00000000

0000 8000 FFFF 8000 0000 7FFF

wvar1 word 0,32768,65535,-32768,0,+32767

相对地址

汇编语句

机器指令

WORD伪指令定义的每个数据都是16位、2个字节

数据段

7FFFH
0000H
8000H
FFFFH
8000H
0000H

+2
wvar1

字变量程序—2

0000000C 0001 FFFF 0026 FFDA 0038 FFC8

wvar2 word 1,-1,38,-38,38h,-38h

1和-1，字节量01H和FFH

字量是0001H和和FFFFH

负数-38H，

字节量补码：C8H(=[1]00H-38H)

字量补码：FFC8H(=[1]0000HH-0038H)

数据段

FFC8H
0038H
FFDAH
0026H
FFFFH
0001H

wvar2
⇒

字变量程序—3

0000000C 0001 FFFF 0026 FFDA 0038 FFC8

wvar2 word 1,-1,38,-38,38h,-38h

000000180000

wvar3 word ?

数据段

wvar3



0000H
FFC8H
0038H
FFDAH
0026H
FFFFH
0001H



WVAR3无初值（被填入0） 占
用16位（2个字节）空间

字变量程序—4

0000001A 2010 1020

wvar4 word 2010h,1020h

=0000000A minint = 10

0000001E 00000005 [000A 0000]

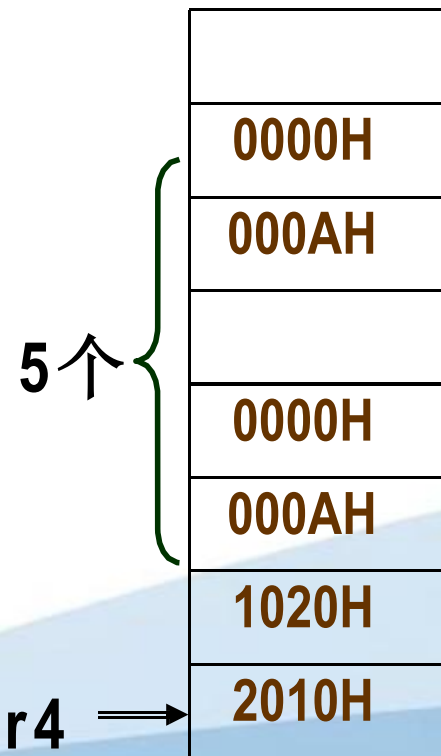
word 5 dup(minint,?)

实际上就是

word 5 dup(10,?)

符号常量使用等价的数值替代

数据段



本讲总结

- 16位变量定义，使用word伪指令
 - 每个数据是一个字量，占用2个存储单元
- 变量定义的参数不区别有无符号
 - 可以是无符号数，也可以是有符号数

对应C语言变量类型: **short**

还有一个问题：16位数据占2个存储单元，哪个在前、哪个在后呢？

32位变量定义

汇编语言的变量定义

➤ 变量需要先定义才能使用

➤ 变量定义的格式

变量名 变量定义伪指令 参数, 参数

➤ 变量定义之后

- 分配了存储空间
 - 赋予了初值（也可以没有初值）
 - 创建了变量名
- （可以获知变量的地址和类型）



主要的变量定义伪指令

助记符: BYTE 变量类型: 字节	分配一个或多个字节单元 每个数据是8位、字节量
助记符: WORD 变量类型: 字	分配一个或多个字单元 每个数据是16位、字量
助记符: DWORD 变量类型: 双字	分配一个或多个双字单元 每个数据是32位、双字量

对应C语言变量类型: **char** **short** **long**



DWORD: 定义双字量(32位)数据的变量

- 32位无符号整数: $0 \sim 2^{32}-1$
- 32位补码表示的有符号整数: $-2^{31} \sim +2^{31}-1$
- 32位逻辑地址 (含16位段地址和16位偏移地址)



32位 (双字量、Doubleword-sized)

```
dvar1    dword 0,80000000h,0ffffffffh  
dvar2    dword ?  
array    dword 5 dup(0)
```


双字变量程序—1

00000000 00000000 80000000 FFFFFFFF

80000000 00000000 7FFFFFFF

dvar1 dword 0,80000000h,0ffffffffh

dword -80000000h,0,7ffffffffh

相对地址

汇编语句

机器指令

DWORD伪指令定义的每个数据都是32位、4个字节

数据段

7FFFFFFFH
00000000H
80000000H
FFFFFFFFH
80000000H
00000000H

dvar1 +4

双字变量程序—2

00000018 00000001 FFFFFFFF 00000026
FFFFFFDA 00000038 FFFFFFFC8

dvar2 dword 1,-1,38,-38,38h,-38h

1和-1，字节量是01H和FH

双字量是00000001H和FFFFFFFFH

负数-38H：字节量补码：：C8H

双字量补码：FFFFFFC8H

数据段

FFFFFFC8H
00000038H
FFFFFFDAH
00000026H
FFFFFFFFH
00000001H

dvar2 ⇒

双字变量程序—3

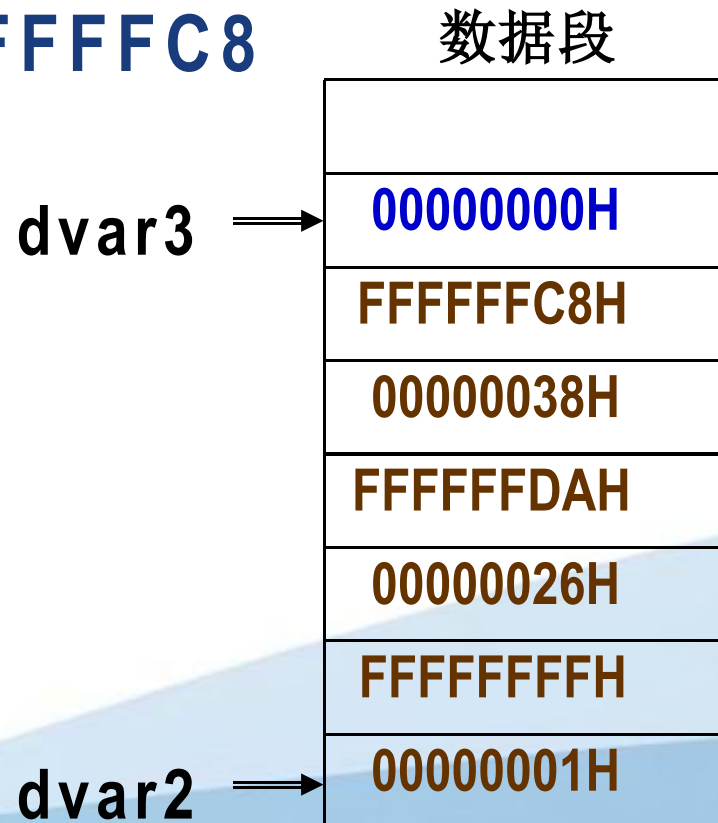
00000018 00000001 FFFFFFFF 00000026
FFFFFFDA 00000038 FFFFFFFC8

dvar2 dword 1,-1,38,-38,38h,-38h

00000030 00000000

dvar3 dword ?

DVAR3无初值（被填入0） 占
用32位（4个字节）空间



双字变量程序—4

00000034 00002010 00001020

dword 2010h,1020h

=0000000A minint = 10

0000003C 0000000A [0000000A 00000000]

dvar5 dword minint dup(minint,?)

10个

实际上就是

dvar5 dword 10 dup(10,?)

数据段

00000000H
0000000AH
00000000H
0000000AH
00001020H
00002010H

dvar5

符号常量使用等价的数值替代



- 32位变量定义，使用dword伪指令
 - ▶ 每个数据是一个双字量，占用4个存储单元
- 变量定义的参数不区别有无符号
 - ▶ 可以是无符号数，也可以是有符号数

对应C语言变量类型: **long**

还有一个问题： 32位数据占4个存储单元，如何安排这4个单元？

多字节数据的存储顺序

字节编址的主存储器

➤ 最小的存储单位：二进制位（比特位bit）

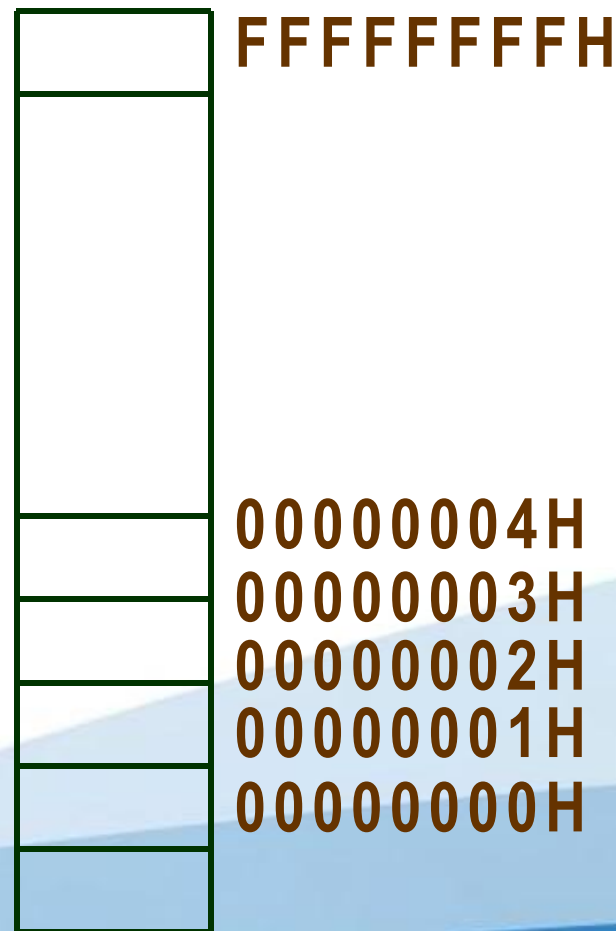
▶ 最常用的存储单位：字节（Byte）

➤ 8个二进制位是一个字节

- ▶ 一个存储单元保存一个字节量数据
- ▶ 一个存储单元对应一个存储器地址

D7 D6 D5 D4 D3 D2 D1 D0

字节



多字节数据的存储顺序

- 变量保存于字节编址的主存储器中
- 每个存储单元保存一个8位、字节量数据
- 多个字节数据顺序逐个存放在主存相邻单元

bvar6 **byte** 39h,31h,32h,38h
wvar6 **word** 3139h,3832h
dvar6 **dword** 38323139h



9128
9128
9128



字节量数据的顺序存储

bvar6 **byte** 39h,31h,32h,38h

高地址 → 03

02

01

低地址 → 00

地址

38H
32H
31H
39H

字量数据的存储顺序

wvar6 word 3139h,3832h



高地址



32H

38H

39H

31H

低地址



大端方式

38H

32H

31H

39H

小端方式

双字量数据的存储顺序

dvar6 dword 38323139h



高地址



39H

31H

32H

38H

低地址



大端方式

38H

32H

31H

39H

小端方式

小端存储和大端存储

➤ 小端方式 (Little Endian)

- 高字节数据保存在高地址存储单元
- 低字节数据保存在低地址存储单元

高对高、低对低

➤ 大端方式 (Big Endian)

- 高字节数据保存在低地址存储单元
- 低字节数据保存在高地址存储单元

高对低、低对高



How to open an egg,
from the little end or the big end ?

80x86采用小端方式存储多字节数据

00405093H	38H
00405092H	32H
00405091H	31H
00405090H	39H
地址	

高对高、低对低

字节量: [00405090H]=39H

字量: [00405090H]=3139H

双字量: [00405090H]=38323139H

本讲总结

- 主存储器采用字节编址
 - ▶ 一个存储单元保存一个字节量数据
 - ▶ 一个存储单元对应一个存储器地址
- 对于N个字节的数据 ($N \geq 2$)
 - ▶ 存储在N个连续的存储单元、具有N个存储器地址
 - ▶ 以最低地址表达该数据地址
 - ▶ 存储采用小端方式：“高对高、低对低”
或者采用大端方式：“高对低、低对高”



变量的地址属性

变量属性

➤ 变量定义 变量名 变量定义伪指令 参数, 参数

➤ 变量定义可以

- ▶ 分配存储空间
- ▶ 赋初值
- ▶ 创建变量名

➤ 定义后的变量名具有两类属性

- (1) 地址属性: 首个变量所在存储单元的逻辑地址
- (2) 类型属性: 变量定义的数据单位

数据段



变量名 →

变量的地址属性及地址操作符

- 变量的地址属性指所在存储单元的逻辑地址
 - 含有段基地址和偏移地址
- 通过地址操作符获得变量的地址属性值

[]	括起的表达式作为存储器地址指针
\$	返回当前偏移地址
OFFSET 变量名	返回变量名所在段的偏移地址
SEG 变量名	返回段基地址（实地址存储模型）

变量地址属性程序

;数据段

```
bvar    byte 12h,34h
        org $+10
array   word 1,2,3,4,5,6,7,8,9,10
wvar    word 5678h
arr_size = $-array
arr_len  = arr_size/2
dvar    dword 9abcdef0h
```

;代码段

```
mov al, bvar
mov ah, bvar+1
mov bx, wvar[2]
mov ecx, arr_len
mov edx, $
mov esi, offset dvar
mov edi, [esi]
mov ebp, dvar
```

变量地址属性程序—5

.data

; 1. bvar byte 12h,34h —— 定义2个字节，占用地址 00000000H ~ 00000001H

bvar byte 12h,34h

; 2. org \$+10 —— \$ 是当前地址（00000002H），+10 表示跳过10个字节

; 占用地址 00000002H ~ 0000000BH（共10字节）

org \$+10

; 3. array word 1,2,...,10 —— 10个word（每个2字节），共20字节

; 占用地址 0000000CH ~ 0000001FH

array word 1,2,3,4,5,6,7,8,9,10

; 4. wvar word 5678h —— 1个word（2字节）

; 占用地址 00000020H ~ 00000021H

wvar word 5678h

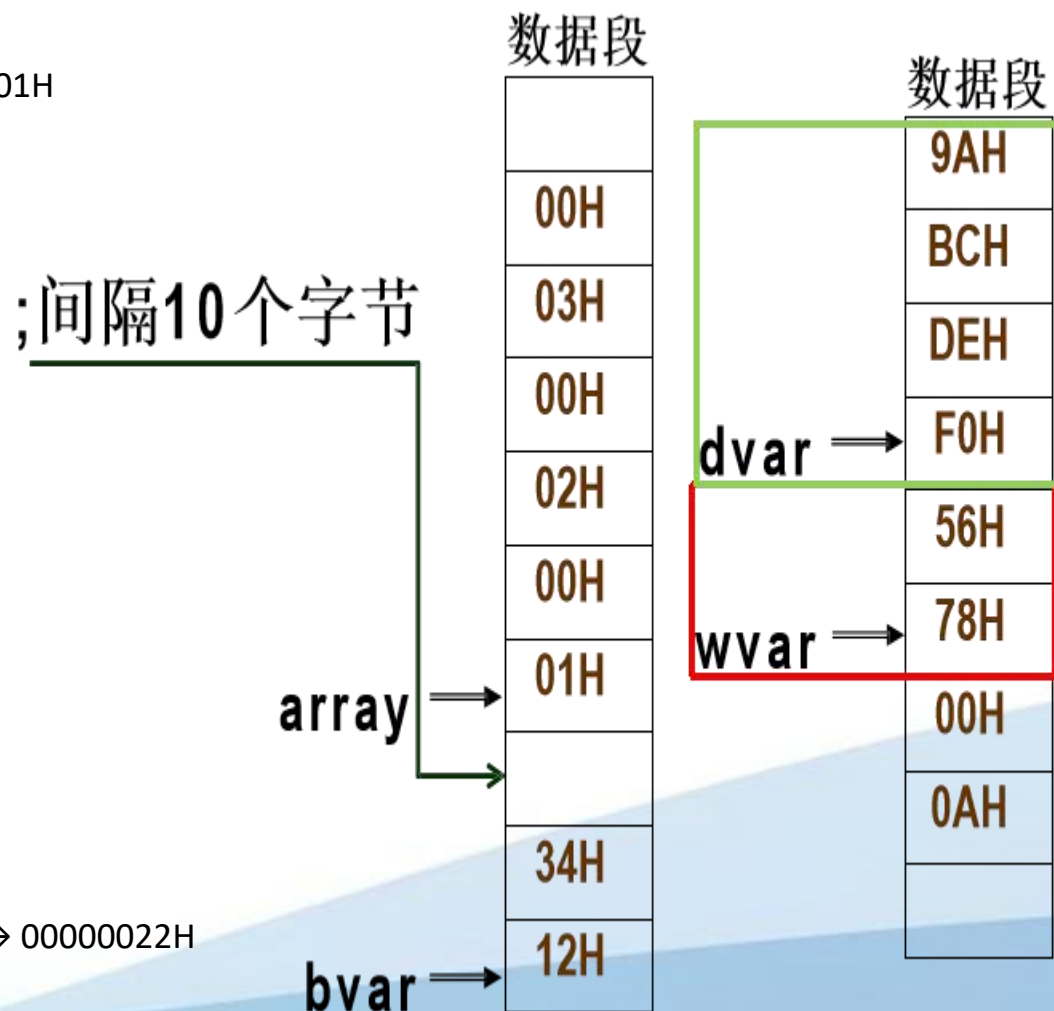
; 5. arr_size/arr_len —— 符号常量，仅汇编阶段计算，不占用内存

arr_size = \$ - array

arr_len = arr_size/2

; 6. dvar dword 9abcdef0h —— 起始地址就是上一个变量的结束地址+1 → 00000022H

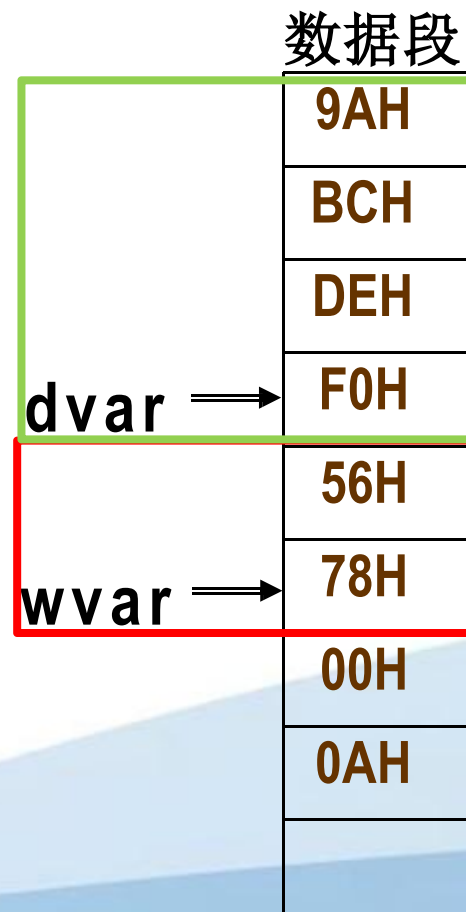
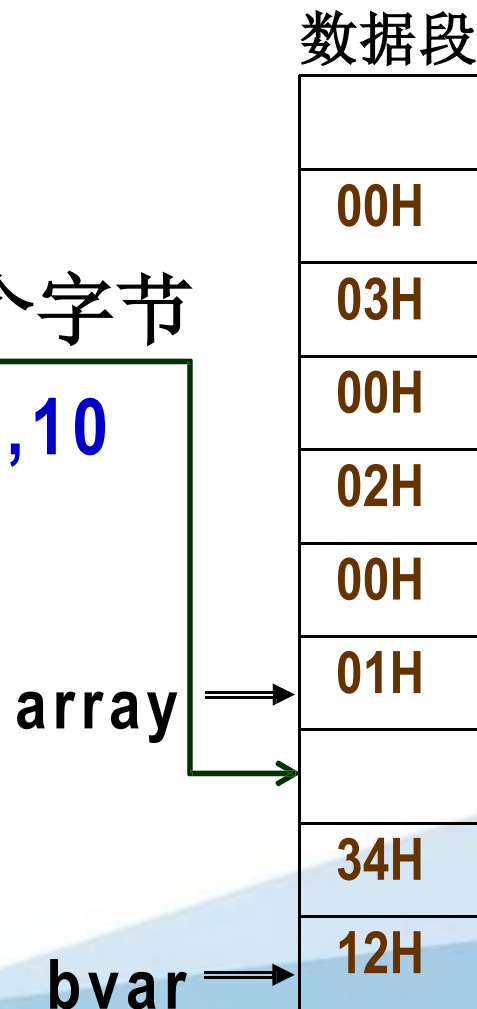
dvar dword 9abcdef0h



变量类型属性程序—数据段

```

;数据段
bvar      byte 12h,34h
          org $+10      ;间隔10个字节
array     word 1,2,3,4,5,6,7,8,9,10
wvar      word 5678h
arr_size  = $-array
arr_len   = arr_size/2
dvar      dword 9abcdef0h
    
```



高位低位别记错！从下到上：低到高，
word是2个存储单元，dword是4个

变量地址属性程序—1

;数据段

机器指令

汇编语句

00000000

12 34

bvar

byte 12h,34h

当前地址 00000002H

相对地址

org \$+10

;当前地址(00000002H)加10，等于0000000CH

0000000C

0001 0002 0003 0004 0005 0006

0007 0008 0009 000A

array

word 1,2,3,4,5,6,7,8,9,10

变量地址属性程序—2

00000020 5678

wvar word 5678h

=00000016 arr_size = \$-array

;计算出当前到ARRAY变量所占存储空间

=0000000B arr_len = arr_size/2

; (除以类型值) 计算出变量个数

00000022 9ABCDEF0

dvar dword 9abcdef0h

当前地址 00000022H

array的地址 0000000CH

数据段使用变量名
代表其偏移地址

变量地址属性程序—3

;代码段

A0 00000000 R

mov al, bvar

;bvar等同于[bvar] 8A 25

00000001 R

mov ah, bvar+1

bvar byte 12h,34h

AL=12H

AH=34H

数据段

+1
bvar

34H

12H

代码段使用变量名

代表其首个数据(变量值)

变量名加减常量

指向首个数据的前后单元



变量地址属性程序—4

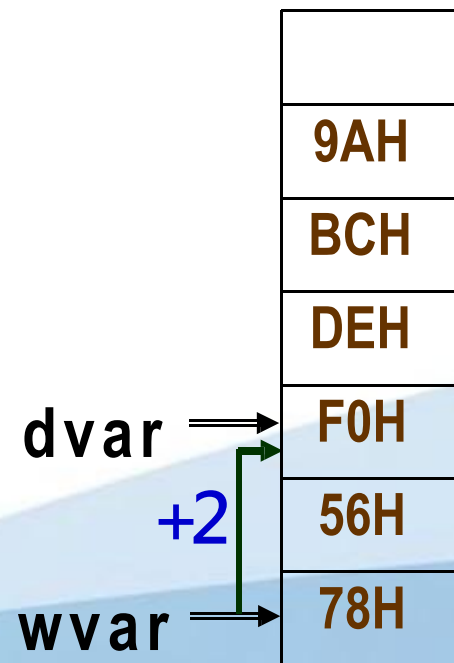
0000000B 66|8B 1D 00000022 R
mov bx, wvar[2]

wvar word 5678h
dvar dword 9abcdef0h

BX=DEF0H

变量名[n] = 变量名+n 常量n表示n个存储单元 指向首个数据的前后单元

数据段





变量地址属性程序—5

00000012 B9 0000000B

mov ecx, arr_len

00000017 BA 00000017 R

mov edx, \$

;\$表示当前（指令）地址

记住！ECX是32位的寄存器，不要漏掉高位的信息

ECX=0000000BH

EDX=代码段地址+17H

计算：\$-array

由于\$和array之间隔着一个wvar，大小是2个字节；

\$-array=array的大小加上wvar的大小；

因此：

arr_size=22=16H

arr_len=11=0BH

=0000000B arr_len = arr_size/2
;（除以类型值）计算出变量个数

变量地址属性程序—6

0000001C BE 00000022 R

ESI=数据段地址+22H

mov esi, offset dvar

00000021 8B 3E

mov edi, [esi]

;通过地址获得变量值

00000022 9ABCDEF0
dvar dword 9abcdef0h

00000023 8B 2D 00000022 R

EDI=9ABCDEF0H

mov ebp, dvar

EBP=9ABCDEF0H

;直接获得变量值

数据段

9AH
BCH
DEH
F0H

dvar →

EDI: 通过“地址 → 内存 → 值”的间接方式, 拿到了 dvar 的值。

EBP: 通过“变量名 → 内存 → 值”的直接方式, 也拿到了 dvar 的值。

因为它们最终读取的是内存中同一个位置 (dvar 的地址) 的同一个数据, 所以结果必然相同。

变量地址属性程序—7

00000029 E8 00000000 E

call disprd

;显示通用寄存器内容

子程序名	DISPRD
功能说明	显示8个32位通用寄存器内容

运行结果

EAX=00003412, EBX=7FFDDEF0, ECX=0000000B, EDX=00401017
ESI=00405022, EDI=9ABCDEF0, EBP=9ABCDEF0, ESP=0013FFC4

AX=3412, BX = DEF0H, EAX和EBX的高16位（0000, 7FFD）是之前遗留的值或调试环境填充值。ECX是array元素的个数加上中间隔着的wvar, ESP是栈指针寄存器, 指向当前栈顶地址, 由系统和调用栈动态决定。EDI和EBP最终读取的是内存中同一个位置（dvar的地址）的同一个数据, 所以结果必然相同。

变量地址属性程序

;数据段

```
bvar    byte 12h,34h
        org $+10
array   word 1,2,3,4,5,6,7,8,9,10
wvar    word 5678h
arr_size = $-array
arr_len  = arr_size/2
dvar    dword 9abcdef0h
```

;代码段

```
mov al, bvar
mov ah, bvar+1
mov bx, wvar[2]
mov ecx, arr_len
mov edx, $
mov esi, offset dvar
mov edi, [esi]
mov ebp, dvar
```



变量的类型属性

变量属性

➤ 变量定义 变量名 变量定义伪指令 参数, 参数

➤ 变量定义可以

- ▶ 分配存储空间
- ▶ 赋初值
- ▶ 创建变量名

➤ 定义后的变量名具有两类属性

- (1) 地址属性: 首个变量所在存储单元的逻辑地址
- (2) 类型属性: 变量定义的数据单位

数据段



变量名 →

变量的类型属性

- 变量的类型属性表示变量定义的数据单位
- 通过类型操作符获得变量的类型属性值

变量定义	类型名	类型值（字节数）
字节变量定义 BYTE	BYTE	1
字变量定义 WORD	WORD	2
双字变量定义 DWORD	DWORD	4

↑
TYPE 变量名

类型操作符

➤ 类型操作符使用变量名的类型属性

类型名 PTR 变量名	将变量名按照指定的类型使用
TYPE 变量名	返回占用字节空间的字量数值
LENGTHOF 变量名	返回整个变量的数据项数
SIZEOF 变量名	返回整个变量占用的字节数

变量类型属性程序

;数据段

```
bvar    byte 12h,34h
        org $+10
array   word 1,2, ,9,10
wvar    word 5678h
arr_size = $-array
arr_len  = arr_size/2
dvar    dword 9abcdef0h
```

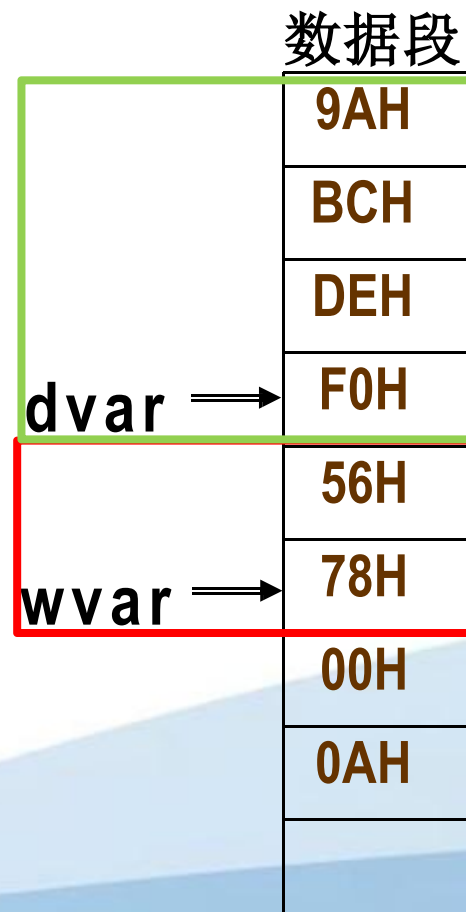
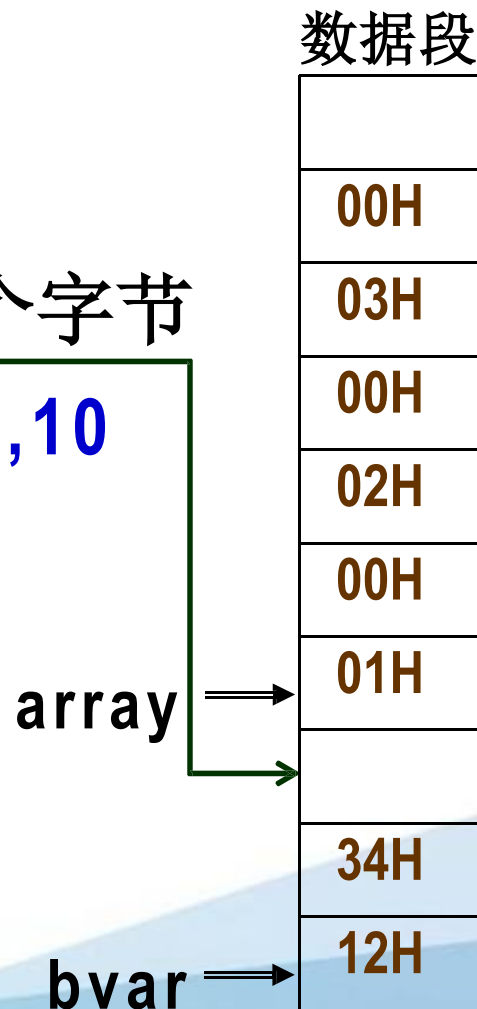
;代码段

```
mov eax, dword ptr array
mov ebx, type bvar
mov ecx, type wvar
mov edx, type dvar
mov esi, lengthof array
mov edi, sizeof array
mov ebp, arr_size
call disprd
```

变量类型属性程序—数据段

```

;数据段
bvar      byte 12h,34h
           org $+10      ;间隔10个字节
array     word 1,2,3,4,5,6,7,8,9,10
wvar      word 5678h
arr_size  = $-array
arr_len   = arr_size/2
dvar      dword 9abcdef0h
    
```



高位低位别记错！从下到上：低到高，
word是2个存储单元，dword是4个

变量类型属性程序一代码段1

;代码段

```
mov eax, dword ptr array
```

被定义为字类型

以双字类型访问

EAX=00020001H

数据段

array 本身被定义为 word（16 位）类型，但 dword ptr array 强制告诉汇编器，将从 array 地址开始的 4 个字节当作一个 32 位的双字（dword）来读取。因为数组的前两个元素是 1 和 2，按小端序存储为 01 00 02 00，组合成一个双字就是 00020001H。

寄存器具有确定的类型属性

8 位寄存器是字节类型 byte

16 位寄存器是字类型 word 32 位寄存器是双字类型 dword

array →

bvar →

00H

03H

00H

02H

00H

01H

34H

12H

变量类型属性程序一代码段2

`mov ebx, type bvar`
;获得字节变量类型值

`EBX=00000001H`

因为 byte 类型
占 1 字节。

`mov ecx, type wvar`
;获得字变量类型值

`ECX=00000002H`

word 类型占 2
字节。

`mov edx, type dvar`
;获得双字变量类型值

`EDX=00000004H`

dword 类型占 4
字节。

类型值就是每个数据所占的字节数 字节类型byte为1、字
类型word为2、双字类型dword为4

变量类型属性程序一代码段3

```
mov esi, lengthof array
```

;获得变量的数据个数（项数）

```
mov edi, sizeof array
```

;获得变量所占的存储空间个数

ESI=0000000AH

EDI=00000014H

;数据段

```
array word 1,2,3,4,5,6,7,8,9,10
```

1个是数量10=0000000AH, 一个是占用20个字节空间=00000014H



变量类型属性程序一代码段4

```
mov ebp, arr_size
```

```
EBP=00000016H
```

```
call disprd
```

;显示寄存器内容

```
array word 1,2,3,4,5,6,7,8,9,10
```

```
wvar word 5678h
```

```
arr_size      = $-array
```

综合前面所有运算
运行结果

```
EAX=00020001, EBX=00000001, ECX=00000002, EDX=00000004
```

```
ESI=0000000A, EDI=00000014, EBP=00000016, ESP=0013FFC4
```

计算：\$-array；由于\$和array之间隔着一个wvar，大小是2个字节；

\$-array=array的大小加上wvar的大小；

因此： arr_size=22=16H

变量类型属性程序

;数据段

```
bvar    byte 12h,34h
        org $+10
array   word 1,2, ,9,10
wvar    word 5678h
arr_size = $-array
arr_len  = arr_size/2
dvar    dword 9abcdef0h
```

;代码段

```
mov eax, dword ptr array
mov ebx, type bvar
mov ecx, type wvar
mov edx, type dvar
mov esi, lengthof array
mov edi, sizeof array
mov ebp, arr_size
call disprd
```




变量程序对比

变量地址属性程序

	;数据段	;代码段
bvar	byte 12h,34h org \$+10	mov al, bvar mov ah, bvar+1
array	word 1,2,3,4,5,6,7,8,9,10	mov bx, wvar[2]
wvar	word 5678h	mov ecx, arr_len
arr_size	= \$-array	mov edx, \$
arr_len	= arr_size/2	mov esi, offset dvar
dvar	dword 9abcdef0h	mov edi, [esi] mov ebp, dvar

变量类型属性程序

	;数据段	;代码段
bvar	byte 12h,34h org \$+10	mov eax, dword ptr array mov ebx, type bvar
array	word 1,2, ,9,10	mov ecx, type wvar
wvar	word 5678h	mov edx, type dvar
arr_size	= \$-array	mov esi, lengthof array
arr_len	= arr_size/2	mov edi, sizeof array
dvar	dword 9abcdef0h	mov ebp, arr_size call disprd

作业

2.1 简答题 (2、3、4、7、8)

2.2 判断题 (2、4、8、9、10)

2.3 填空题 (1、2、3、8、9)

2.10、2.14、2.15、2.17